

03-classes

June 12, 2022

```
[1]: # See attributes and methods of type str object
# Methods are associated to objects, functions are not
myString = "a string"
dir(myString)
```

```
[1]: ['__add__',
      '__class__',
      '__contains__',
      '__delattr__',
      '__dir__',
      '__doc__',
      '__eq__',
      '__format__',
      '__ge__',
      '__getattr__',
      '__getitem__',
      '__getnewargs__',
      '__gt__',
      '__hash__',
      '__init__',
      '__init_subclass__',
      '__iter__',
      '__le__',
      '__len__',
      '__lt__',
      '__mod__',
      '__mul__',
      '__ne__',
      '__new__',
      '__reduce__',
      '__reduce_ex__',
      '__repr__',
      '__rmod__',
      '__rmul__',
      '__setattr__',
      '__sizeof__',
      '__str__']
```

'__subclasshook__',
'capitalize',
'casefold',
'center',
'count',
'encode',
'endswith',
'expandtabs',
'find',
'format',
'format_map',
'index',
'isalnum',
'isalpha',
'isascii',
'isdecimal',
'isdigit',
'isidentifier',
'islower',
'isnumeric',
'isprintable',
'isspace',
'istitle',
'isupper',
'join',
'ljust',
'lower',
'lstrip',
'maketrans',
'partition',
'removeprefix',
'removesuffix',
'replace',
'rfind',
'rindex',
'rjust',
'rpartition',
'rsplit',
'rstrip',
'split',
'splitlines',
'startswith',
'strip',
'swapcase',
'title',
'translate',
'upper',

```
'zfill']
```

```
[2]: class Person:
      def __dir__(self):
          return ['age', 'name', 'salary']

      teacher = Person()
      dir(teacher)
```

```
[2]: ['age', 'name', 'salary']
```

0.0.1 What is up with the `__dir__` naming convention?

<https://docs.python.org/3/reference/datamodel.html#special-method-names>

```
[3]: class Cat:
      def __init__(self, name, age):
          self.name = name
          self.age = age

      def info(self):
          print(f"I am a cat. My name is {self.name}. I am {self.age} years old.")

      def make_sound(self):
          print("Meow")
```

0.0.2 A class is a blueprint for building objects.

- Our “Cat” class has attributes “name” and “age”.
- It also has methods “info” and “make_sound”

```
[4]: cat1 = Cat('Andy', 2)
      cat2 = Cat('Phoebe', 3)
```

```
[5]: cat1.info()
```

```
[5]: 'Andy'
```

```
[6]: cat2.make_sound()
```

Meow

0.0.3 The “self” keyword helps python internally keep objects separate.

```
[7]: print(cat1)
```

```
<__main__.Cat object at 0x7fa53c4c1930>
```

```
[8]: print(cat2)
```

```
<__main__.Cat object at 0x7fa53c4c0160>
```